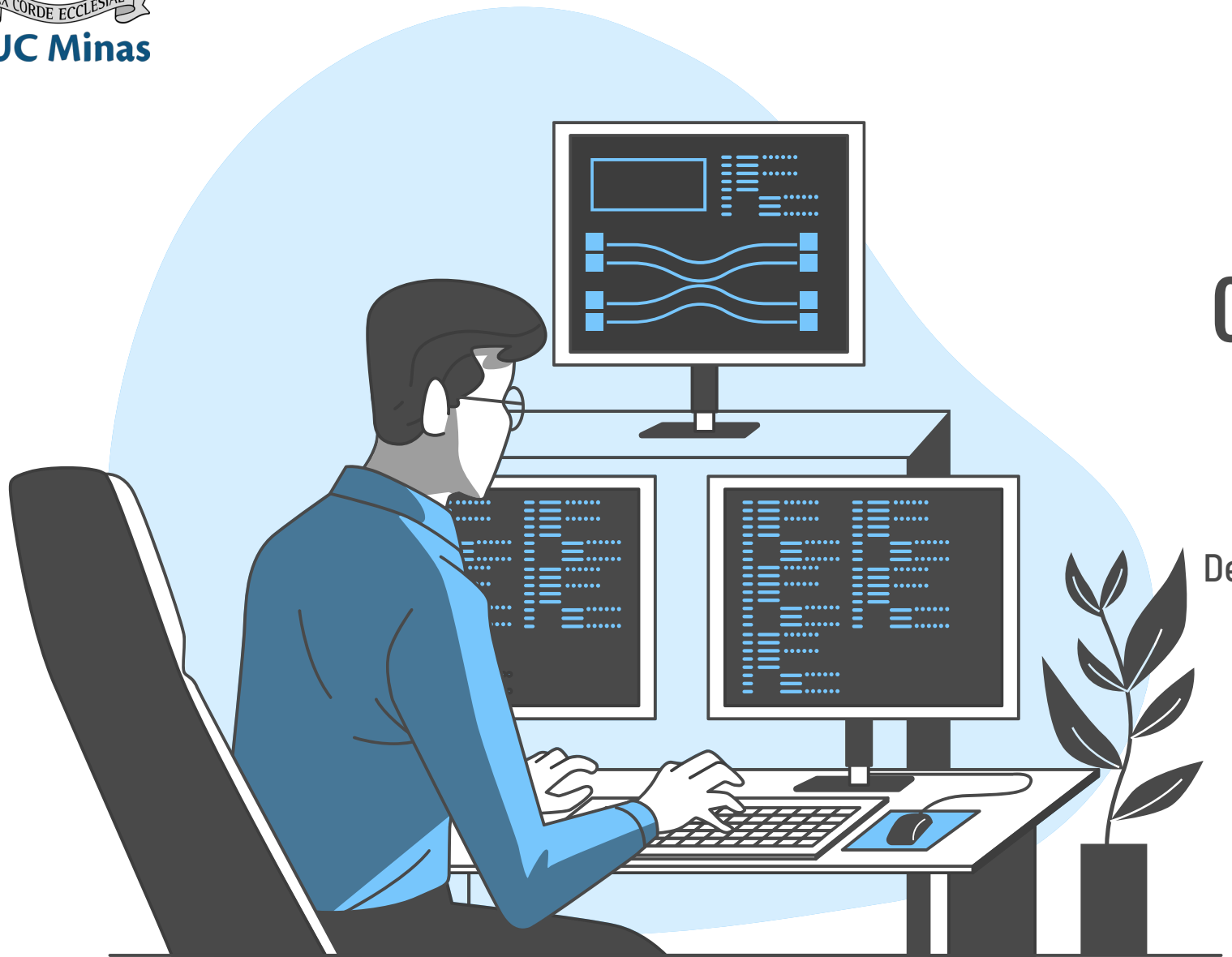
PUC Minas



Aula 09 · Deploy

CI/CD com GitHub Actions

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender **integração contínua** (CI) e **entrega contínua** (CD)
2. Ler a estrutura de um **workflow** do GitHub Actions
3. Rodar os testes do **Spring Boot** com JDK 25 e do **Vite** com Node a cada push
4. Construir a **imagem Docker** no CI
5. Disparar o deploy do Render com **Deploy Hook** e usar **secrets**

Antes de começar

- FullStack, aula 09 (testes)
- Aulas 05 e 06 desta pasta (Vercel e Render)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 CI e CD

02 Anatomia de um workflow

03 Acompanhando a execução

04 Proteção da branch main

05 Construindo a imagem Docker no CI

06 Deploy só depois dos testes

07 Aprofundando: workflows melhores

01

CI e CD

CI e CD

- **Integração contínua (CI)**: a cada push ou pull request, um servidor baixa o código, compila e **roda os testes**. Se algo quebrou, você descobre em minutos, e não no dia da apresentação.
- **Entrega/deploy contínuo (CD)**: se o CI passou, a nova versão é **publicada automaticamente**.

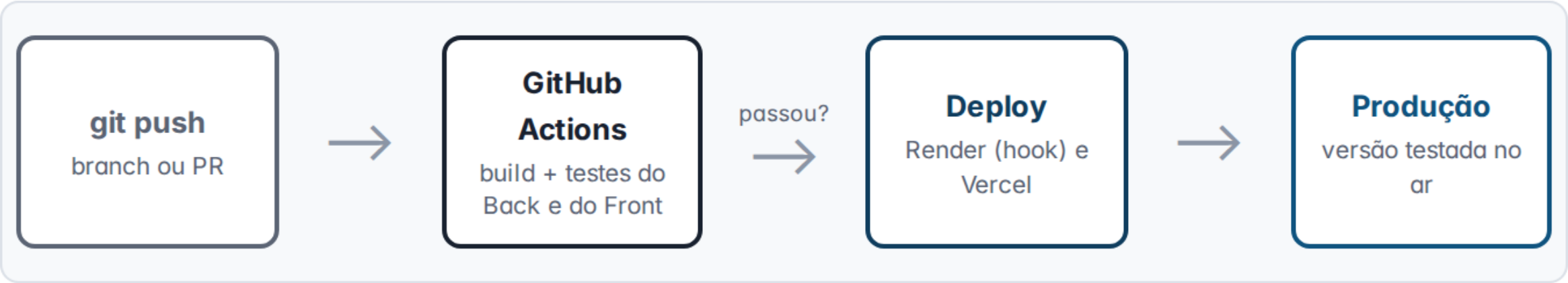
O Vercel e o Render já fazem um tipo de CD: publicam a cada push na `main`. O que falta é a **barreira**: impedir que código com testes quebrados chegue à produção. É isso que o **GitHub Actions** acrescenta.

01

CI e CD

Aula 09 · Deploy

CI e CD



Pipeline de CI/CD: só chega à produção o que passou nos testes.

02

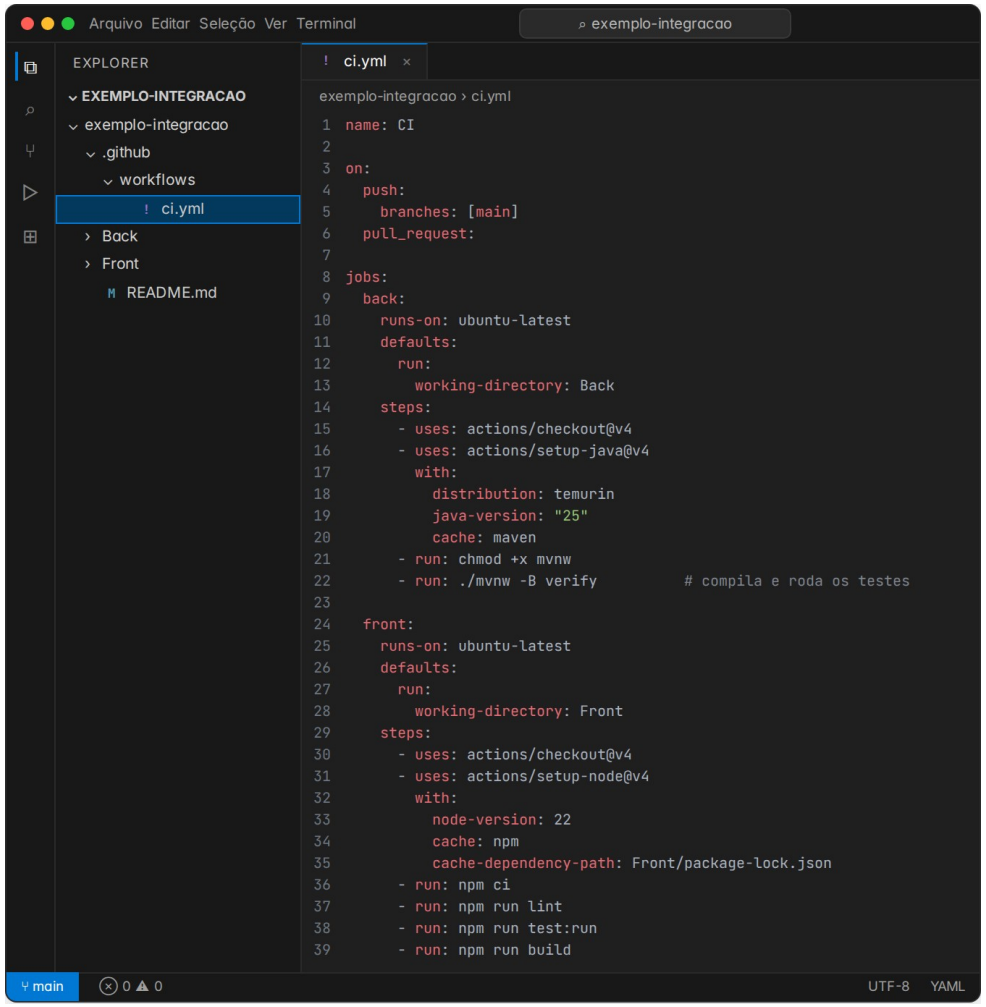
Anatomia de um workflow

Anatomia de um workflow

Um workflow é um arquivo YAML em `.github/workflows/` na **raiz** do repositório. O GitHub detecta o arquivo sozinho.

Chave	Significado
`name`	Nome que aparece na aba Actions
`on`	Quando roda: `push`, `pull_request`, `workflow_dispatch` (botão manual), `schedule`
`jobs`	Tarefas; rodam em paralelo, a não ser que uma dependa da outra (`needs`)
`runs-on`	Máquina usada: `ubuntu-latest`
`steps`	Passos de um job: `uses` (ação pronta) ou `run` (comando)
`working-directory`	Pasta onde os comandos rodam (`Back` ou `Front`)

Anatomia de um workflow



Workflow com dois jobs em paralelo: Back (JDK 25 + Maven) e Front (Node 22 + Vite).

Anatomia de um workflow

Passo	O que faz
`actions/checkout@v4`	Baixa o código do repositório na máquina do CI
`actions/setup-java@v4`	Instala o JDK (Temurin 25) e guarda o cache do Maven
`./mvnw -B verify`	Compila, roda os testes e verifica; `-B` = modo não interativo
`actions/setup-node@v4`	Instala o Node e guarda o cache do npm
`npm ci`	Instalação exata pelo `package-lock.json`
`npm run build`	Garante que o build de produção não quebrou

Anatomia de um workflow

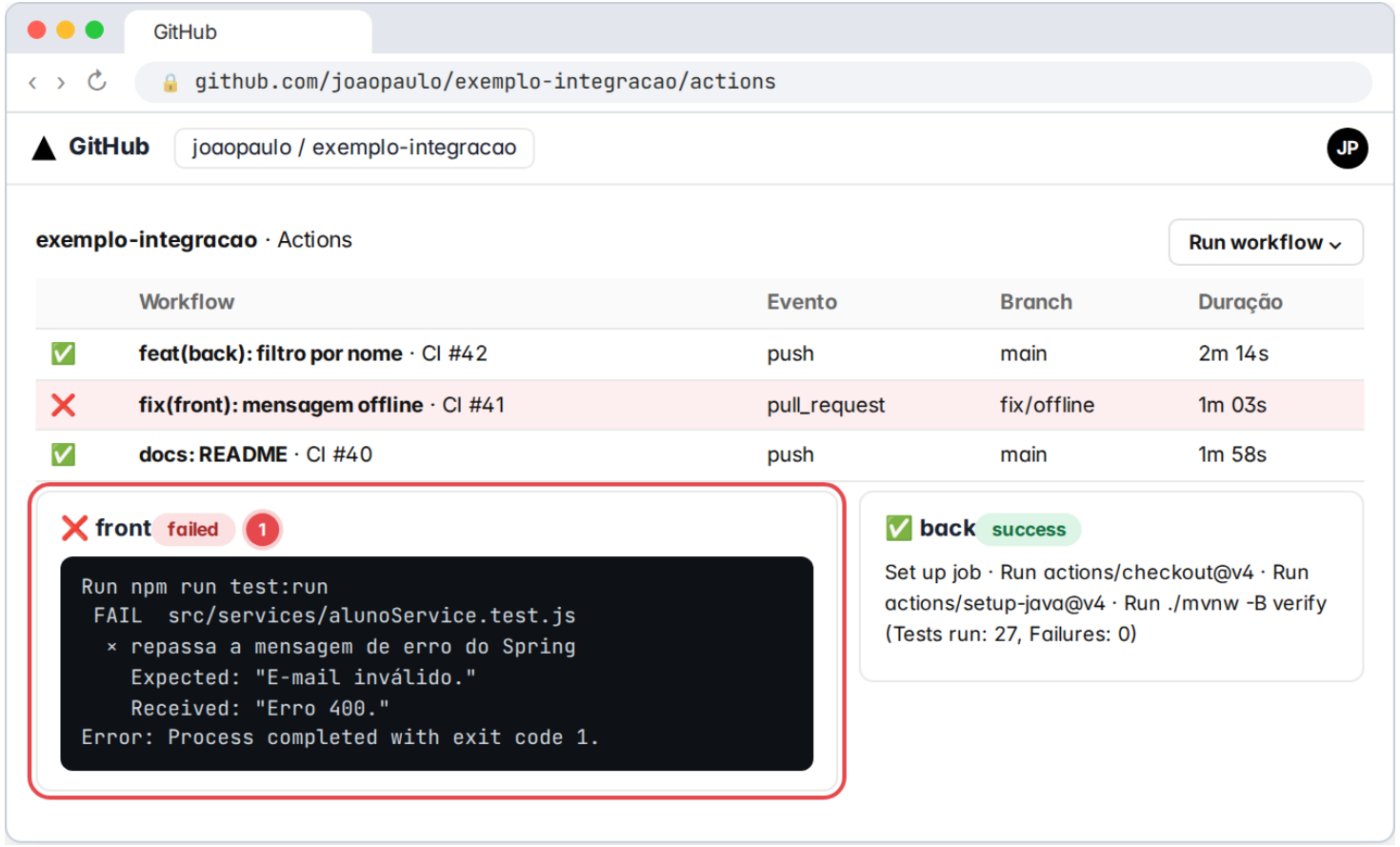
Atenção

Testes que precisam de banco real (MongoDB Atlas, Supabase) falham no CI, porque a máquina do GitHub não tem as credenciais e o Atlas pode bloquear o IP. Mantenha os testes do CI com **mocks** (`@WebMvcTest`, Mockito) ou use **Testcontainers**, que sobe um banco em Docker dentro do próprio CI.

03

Acompanhando a execução

Acompanhando a execução



Aba **Actions**: cada push vira uma execução. Clique em um job para ver o log do passo que falhou.

Acompanhando a execução

1. Abra a aba **Actions** do repositório. Cada linha é uma execução, com o commit que a disparou.
2. Um ❌ indica falha. Clique na execução e depois no job (`front` ou `back`).
3. Expanda o passo vermelho: o log mostra o mesmo erro que você veria rodando o comando no seu computador.
4. Corrija localmente, rode o mesmo comando (`npm run test:run`, `./mvnw verify`) e faça um novo push.

04

Proteção da branch main

Proteção da branch main

Em **Settings → Branches → Add branch ruleset** (ou *branch protection rule*), marque **Require status checks to pass** e selecione os jobs `back` e `front`. A partir daí, um pull request com testes quebrados **não pode** ser mesclado na `main`, e portanto nunca chega ao Render nem ao Vercel.

04

Proteção da branch main

Aula 09 · Deploy

05

Construindo a imagem Docker no CI

Construindo a imagem Docker no CI

Construir a imagem no CI garante que o `Dockerfile` continua funcionando antes de o Render tentar.

.github/workflows/ci.yml (job extra)

```yaml
  docker:
    runs-on: ubuntu-latest
    needs: back                        # só roda se o job back passou
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v6
        with:
          context: ./Back
          push: false                  # só constrói; o Render faz o próprio build
          tags: exemplo-integracao-back:ci
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

06

Deploy só depois dos testes

Deploy só depois dos testes

Por padrão o Render publica a cada push (*Auto-Deploy: On Commit*). Para que ele espere o CI:

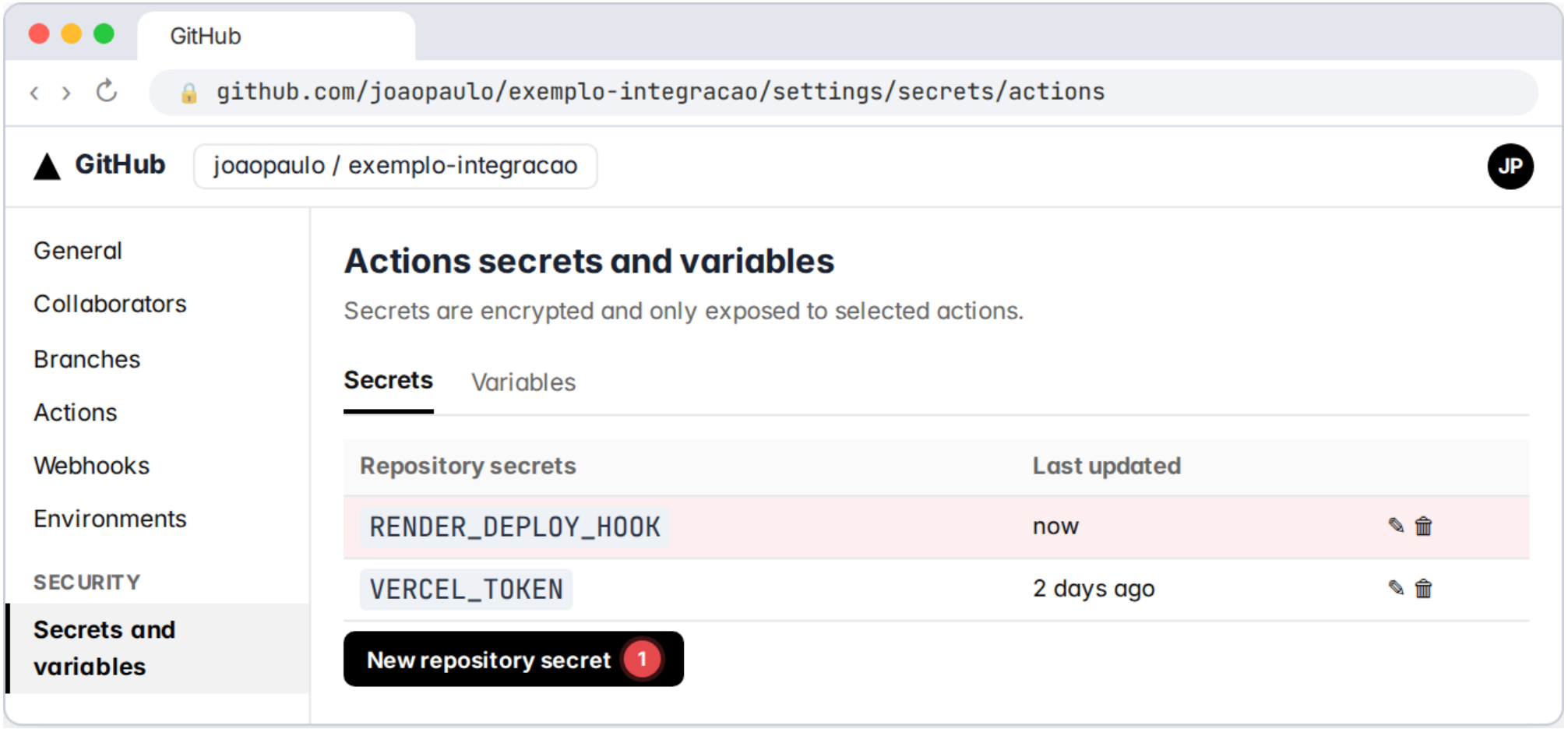
1. No Render, em **Settings → Build & Deploy**, mude **Auto-Deploy** para **After CI Checks Pass** (o Render passa a esperar os checks do GitHub). Ou desligue o Auto-Deploy e use um Deploy Hook.
2. Para o hook: em **Settings → Deploy Hook**, copie a URL secreta.
3. No GitHub, em **Settings → Secrets and variables → Actions → New repository secret**, crie `RENDER_DEPLOY_HOOK` com essa URL.
4. Adicione ao workflow um job `deploy` com `needs: [back, front]` que chama o hook com `curl`.

Deploy só depois dos testes

.github/workflows/ci.yml (job de deploy)

```yaml
  deploy:
    runs-on: ubuntu-latest
    needs: [back, front]
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    steps:
      - name: Dispara o deploy do back no Render
        run: curl -fsS -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
```

Deploy só depois dos testes



Secrets do repositório: o valor fica criptografado e aparece como `***` nos logs.

Deploy só depois dos testes

Observação

O Vercel também integra com os checks do GitHub: em *Settings* → *Git* você pode exigir que os checks passem, e os *Preview Deployments* de cada pull request aparecem como comentário no próprio PR. Para controle total, há a Vercel CLI (`vercel deploy --prod --token`) dentro do workflow.

07

Aprofundando: workflows melhores

Aprofundando: workflows melhores

Disparar só quando a pasta muda

```yaml
on:
  push:
    branches: [main]
    paths: ["Back/**", ".github/workflows/back.yml"]   # só quando o back mudar
  pull_request:
    paths: ["Back/**"]
  workflow_dispatch:                                   # botão "Run workflow"

concurrency:                                           # cancela execução antiga da mesma branch
  group: back-${{ github.ref }}
  cancel-in-progress: true
```

Um workflow x um por pasta

Um `ci.yml` com dois jobs

- Simples de ler
- Roda tudo a cada push
- Bom para projetos pequenos

vs

`back.yml` e `front.yml` com `paths`

- Cada parte só roda quando muda
- CI mais rápido
- Bom quando o projeto cresce

Aprofundando: workflows melhores

Guardando os relatórios de teste

```yaml
      - uses: actions/upload-artifact@v4
        if: always()                    # mesmo se os testes falharem
        with:
          name: relatorio-testes-back
          path: Back/target/surefire-reports/
```

Aprofundando: workflows melhores

Contexto / expressão	Valor
`${{ github.ref }}`	`refs/heads/main`
`${{ github.sha }}`	Hash do commit
`${{ github.event_name }}`	`push`, `pull_request`…
`${{ secrets.NOME }}`	Valor de um secret
`${{ vars.NOME }}`	Valor de uma variável (não secreta)

Badges e boas práticas

Badge no README

`![CI](https://github.com///actions/workflows/ci.yml/badge.svg)` mostra se a main está verde.

Fixe as versões das ações

`@v4` em vez de `@main`: a ação não muda sem você saber.

Menor permissão

`permissions: contents: read` no topo do workflow.

Nunca imprima secrets

`echo $SECRET` aparece mascarado, mas evite mesmo assim.

Aprofundando: workflows melhores

Na prática

Os badges no topo dos README dos projetos da disciplina (Java, Spring Boot, Docker, Vercel, Render) são do serviço *shields.io*. O badge do GitHub Actions é diferente: ele é **dinâmico** e fica vermelho quando o último CI falhou.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

O que é integração contínua?

A Publicar manualmente

B Build e testes automáticos a cada push ou PR

C Usar o Docker

D Um banco de dados

Resposta

O que é integração contínua?

A Publicar manualmente

✓ Build e testes automáticos a cada push ou PR

C Usar o Docker

D Um banco de dados

Por quê: CD publica o que passou.

Pergunta

Onde ficam os workflows do GitHub Actions?

A .github/workflows/

B Back/src/test

C Front/public

D .vscode/

Resposta

Onde ficam os workflows do GitHub Actions?

✓ .github/workflows/

B Back/src/test

C Front/public

D .vscode/

Por quê: Na raiz do repositório.

Pergunta

Qual ação instala o JDK 25 no workflow?

A actions/checkout

B actions/setup-java com java-version 25

C actions/setup-node

D docker/build-push-action

Resposta

Qual ação instala o JDK 25 no workflow?

A actions/checkout

✓ actions/setup-java com java-version 25

C actions/setup-node

D docker/build-push-action

Por quê: Com distribution: temurin e cache: maven.

Pergunta

Como impedir merge na main com testes quebrados?

A Apagando a branch

B Regra de proteção exigindo os status checks

C Desligando o Actions

D Usando o Render

Resposta

Como impedir merge na main com testes quebrados?

A Apagando a branch

✓ Regra de proteção exigindo os status checks

C Desligando o Actions

D Usando o Render

Por quê: Settings → Branches.

Pergunta

Onde guardar a URL do Deploy Hook do Render?

A. No README

B. Em um secret do repositório

C. No ci.yml em texto

D. No package.json

Resposta

Onde guardar a URL do Deploy Hook do Render?

A. No README

✓ Em um secret do repositório

C. No ci.yml em texto

D. No package.json

Por quê: Use ${{ secrets.RENDER_DEPLOY_HOOK }}.

Fechamento

Resumo da aula

- ✓ CI roda build e testes a cada push/PR; CD publica só o que passou.
- ✓ Workflows ficam em `.github/workflows/*.yml`: `on`, `jobs`, `runs-on`, `steps`.
- ✓ Back: `setup-java` (temurin 25, cache maven) + `./mvnw -B verify`. Front: `setup-node` + `npm ci`, lint, testes e build.
- ✓ Proteja a `main` exigindo os checks; testes do CI usam mocks ou Testcontainers.
- ✓ Render: *After CI Checks Pass* ou Deploy Hook guardado em **secrets**.

Exercícios

1. Crie o `ci.yml` com os jobs `back` e `front` no seu repositório e faça o primeiro run ficar verde.

2. Quebre um teste de propósito, abra um pull request e mostre o check vermelho bloqueando o merge.

3. Adicione o job `docker` que constrói a imagem do back.

4. Configure o Deploy Hook do Render como secret e o job `deploy` que só roda na `main`.

Para ir além

- docs.github.com/actions
- github.com/actions/setup-java
- render.com/docs/deploy-hooks

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 09?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web